

Geospatial data analysis in Python

Focus on geoutils + xdem packages

Amaury Dehecq, Romain Hugonnet

Institut des Geosciences de l'Environnement (IGE), Université Grenoble Alpes, IRD, Grenoble, FR



What will you learn today?

- Quick intro to Python geospatial data ecosystem
- Discover two great Python modules: GeoUtils + xDEM ! 🥰
- Learn how to easily open and analyze geospatial data (raster, vector)
- Work with Digital Elevation Models (DEMs), apply coregistration and calculate/plot statistics - optional
- Create easy plots and even interactive plots with Folium

Getting to know you

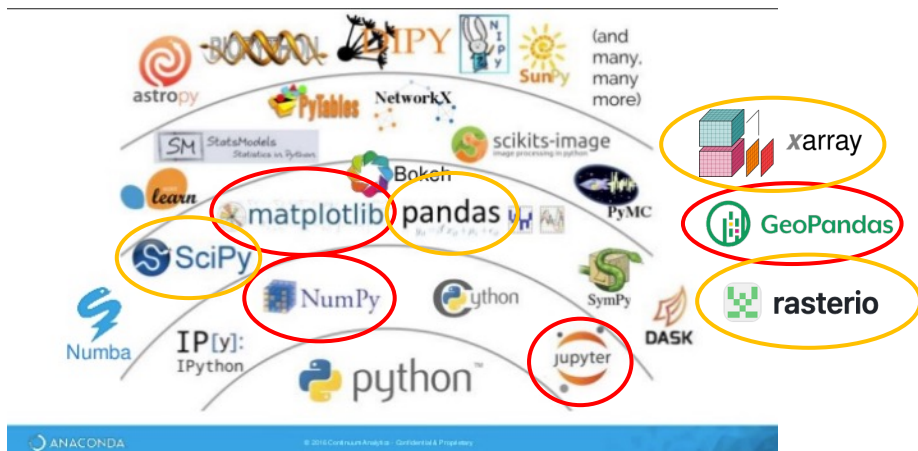
- Are you familiar with Python?
- Have you heard of GDAL?
- Have you worked with rasterio?
- Have you worked with geopandas?
- Have you worked with geoutils/xdem?

Introduction to Python



What is Python?

- A general purpose **programming language, free and open source** (unlike MatLab)
- Interpreted → the code is run directly, without compilation (like MatLab/R; unlike C/Fortran)
- High-level and simple syntax
- Modular and strong community development → many libraries (or modules) are developed on top of Python (NumPy, Matplotlib)



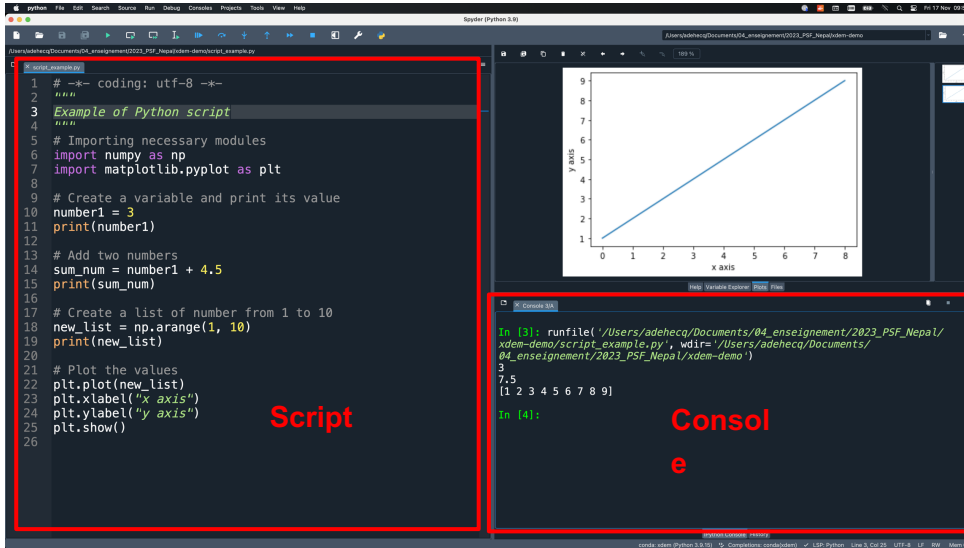
Introduction to Python



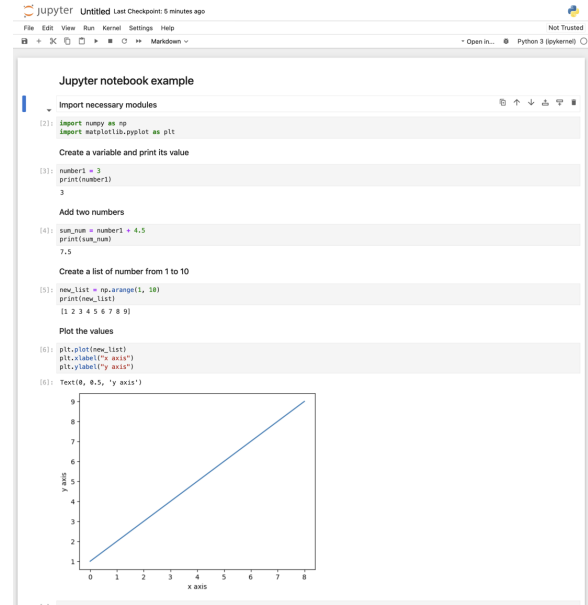
How to run Python code?

- Script (text file) + console, in an IDE*, e.g. Spyder
- Jupyter notebook: can mix text, code and figures/tables → what we will use

Python IDE* Spyder



Jupyter notebook



Introduction to Python



How to install it?

Simplest is to use **Miniconda** → demo

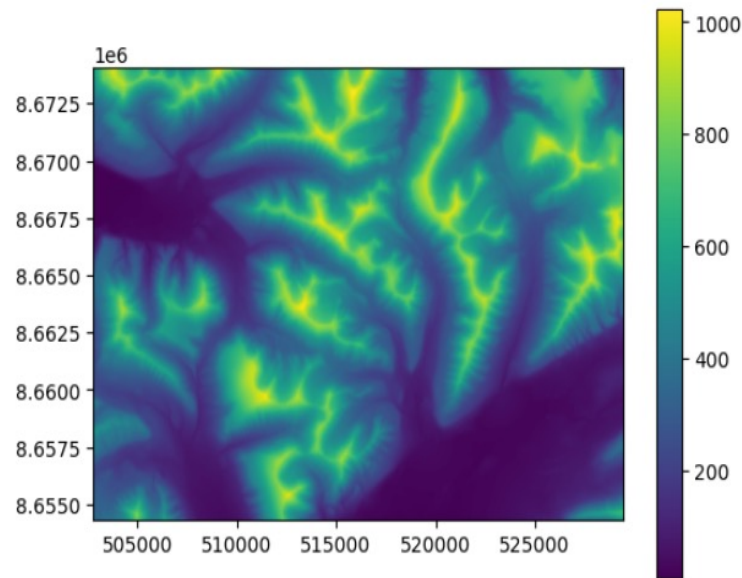
- Download Miniconda: <https://conda.io/projects/conda/en/latest/user-guide/install/windows.html>
- Open Anaconda prompt
- Type ``conda install -n xdem -c conda-forge --strict-channel-priority xdem jupyter``
- Type ``conda activate xdem``
- To open a notebook, type ``jupyter notebook``

Python tutorials to get started: <https://www.learnpython.org/>

Intro to georeferenced data

Raster data (e.g., GTiff)

- Variable represented on a **regular grid**
 - e.g. elevation, temperature...
 - one pixel → one value at a given location
- Data: array (matrix) – 2 or 3D
- Georeferences:
 - Coordinate Reference System (CRS) or projection
 - raster width/height
 - extent or bounds



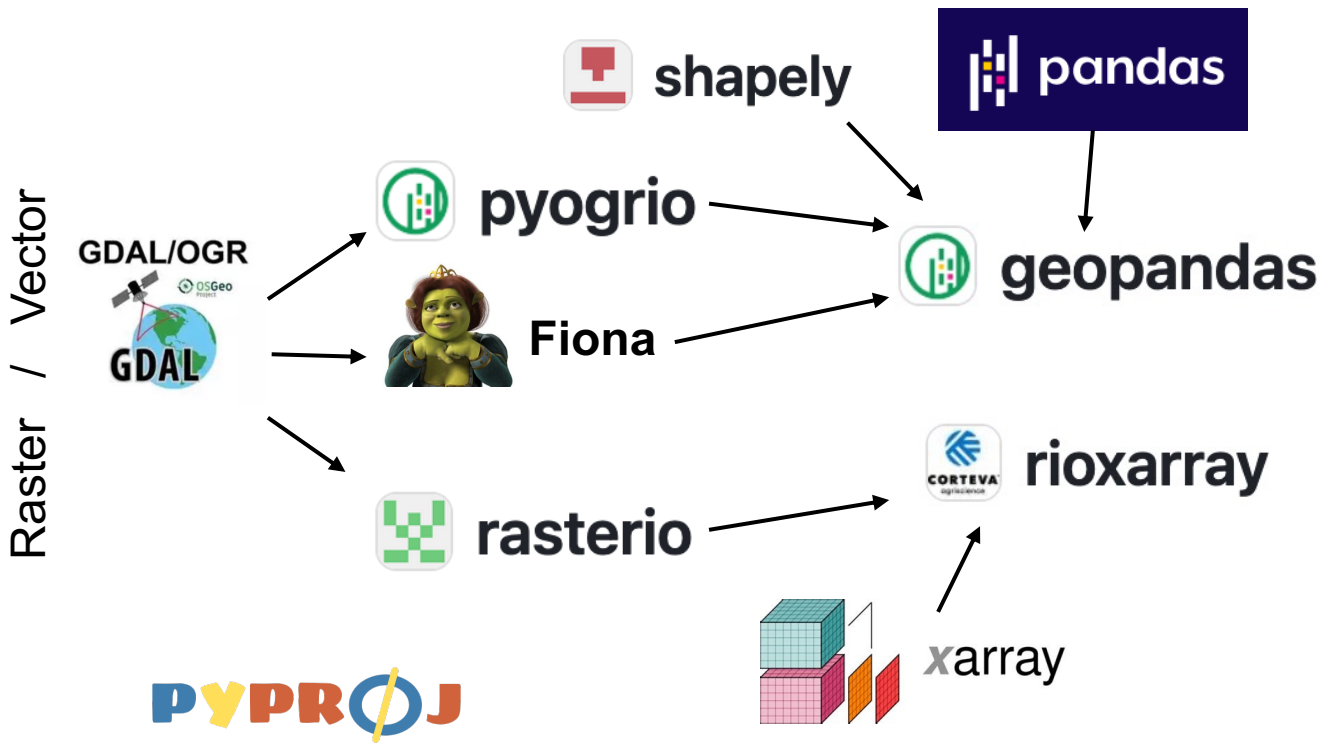
Intro to georeferenced data

Vector data (e.g. Shapefile)

- Objects are represented as **features**: points, lines, polygons (e.g. glacier contour, GPS measurement...)
- Data: list of vertices + attributes (e.g. date, area...)
- Georeferences:
 - Coordinate Reference System



Python geospatial ecosystem



Python geospatial ecosystem

You **do not want** to work with GDAL anymore...

Example of reading a raster band and georeferences in gdal

```
from osgeo import gdal
dataset = gdal.Open(filename, gdal.GA_ReadOnly)
if not dataset:

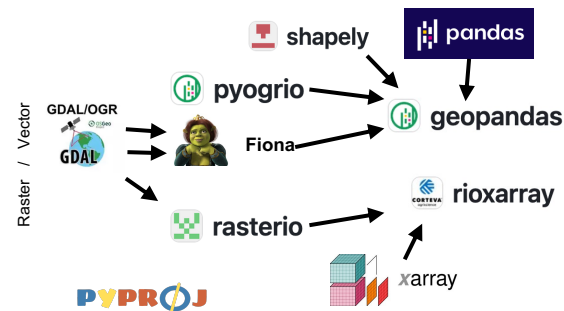
print("Driver: {}/{}".format(dataset.GetDriver().ShortName,
                             dataset.GetDriver().LongName))
print("Size is {} x {} x {}".format(dataset.RasterXSize,
                                     dataset.RasterYSize,
                                     dataset.RasterCount))
print("Projection is {}".format(dataset.GetProjection()))
geotransform = dataset.GetGeoTransform()
if geotransform:
    print("Origin = ({}, {})".format(geotransform[0], geotransform[3]))
    print("Pixel Size = ({}, {})".format(geotransform[1], geotransform[5]))

band = dataset.GetRasterBand(1)
print("Band Type={}".format(gdal.GetDataTypeName(band.DataType)))

min = band.GetMinimum()
max = band.GetMaximum()
if not min or not max:
    (min,max) = band.ComputeRasterMinMax(True)
print("Min={:.3f}, Max={:.3f}".format(min,max))
```

Python geospatial ecosystem

- GDAL/OGR: behind everything you have probably ever used for GS data
- pyproj: Python interface to PROJ = cartographic projections and coordinate transformations library
- Fiona: Better Python interface for OGR
- rasterio: Better Python interface for GDAL
- shapely: For handling planar geometric objects (points, lines, polygons)
- geopandas: Fusion of Fiona (for IO), shapely (for geometries) and pandas (for the data structure) → GeoDataFrame
- pyogrio: faster IO for vector files (compared to Fiona), developed by geopandas
- rioxarray: xarray extension powered by rasterio



Side note: when you don't need Python (or QGIS)

GDAL command lines:

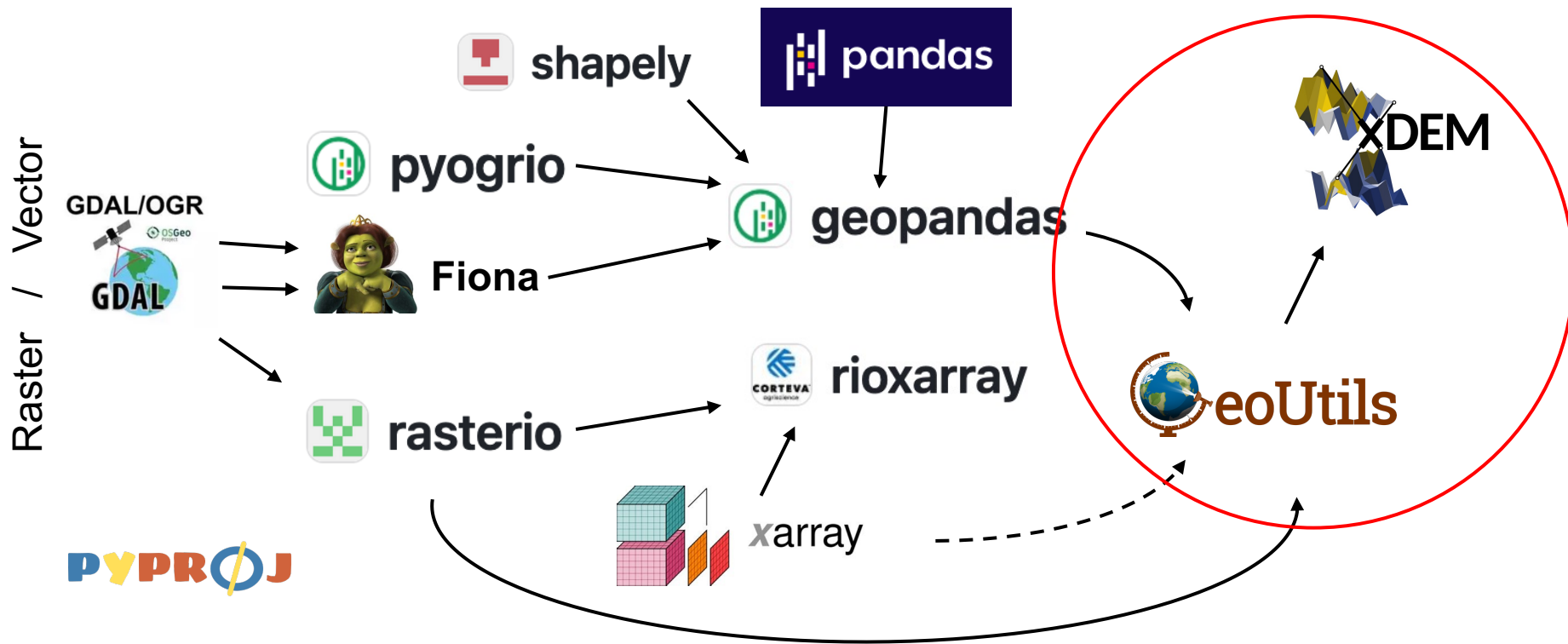
<https://gdal.org/programs/index.html>

- **gdalinfo** → get quick info on a raster file
- **gdalwarp** → reproject/resample a raster
- **gdal_translate** → convert data format, type etc (can be done with gdalwarp)
- **gdal_edit** → edit in place
- **gdalbuildvrt** → creates a virtual dataset (great for creating mosaics or “in the fly” reprojection)
- **gdaldem** → for making hillshades, slope, aspect maps

```
gdalinfo ~/openerv/utm.tif
Driver: GTiff/GeoTIFF
Size is 512, 512
Coordinate System is:
PROJCS["NAD27 / UTM zone 11N",
  GEOGCS["NAD27",
    DATUM["North_American_Datum_1927",
      SPHEROID["Clarke 1866",6378206.4,294.978698213901]],
    PRIMEM["Greenwich",0],
    UNIT["degree",0.0174532925199433]],
  PROJECTION["Transverse_Mercator"],
  PARAMETER["latitude_of_origin",0],
  PARAMETER["central_meridian",-117],
  PARAMETER["scale_factor",0.9996],
  PARAMETER["false_easting",500000],
  PARAMETER["false_northing",0],
  UNIT["metre",1]]
Origin = (440720.000000,3751320.000000)
Pixel Size = (60.000000,-60.000000)
Corner Coordinates:
Upper Left ( 440720.000, 3751320.000) (117d38'28.21"W, 33d54'8.47"N)
Lower Left ( 440720.000, 3720600.000) (117d38'20.79"W, 33d37'31.04"N)
Upper Right ( 471440.000, 3751320.000) (117d18'32.07"W, 33d54'13.08"N)
Lower Right ( 471440.000, 3720600.000) (117d18'28.50"W, 33d37'35.61"N)
Center ( 456080.000, 3735960.000) (117d28'27.39"W, 33d45'52.46"N)
Band 1 Block=512x16 Type=Byte, ColorInterp=Gray
```

Python geospatial ecosystem

This is where we come in!

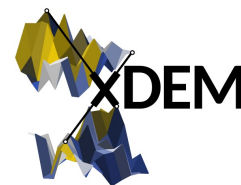


Introduction to GeoUtils/xDEM

- Two Python modules, developed by glaciologists, for geoscientists.
- Core developers: R. Hugonnet (U. of Washington, USA), E. Mannerfelt (U. of Oslo, NO), A. Tedstone (U. of Fribourg, CH), A. Dehecq (IGE, FR)



- high-level functionalities at the interface of raster, vector and point cloud objects
- **IO**: open, save, export
- **Transform**: reproject, crop, translate, merge
- **Raster/vector interface**: rasterize, polygonize, point interpolation, gridding...
- <https://geoutils.readthedocs.io>
- <https://github.com/GlacioHack/GeoUtils/>



- Vertical referencing (datum change)
- Terrain attributes: slope, aspect, hillshade etc.
- Coregistration
- Differencing and volume change
- Spatial statistics, uncertainty analysis
- <https://xdem.readthedocs.io>
- <https://github.com/GlacioHack/xdem/>

Introduction to GeoUtils/xDEM

- **To get started** – browse the great documentation.
- **For active users** – discover the GitHub pages (issues, discussions)
- **Interested in contributing?** Discover how to write collaborative code (pull requests, code review, pre-commits, tests...)

Introduction to GeoUtils/xDEM

Project started in early 2021, with a *GlacioHack* event.

Why the need for geoutils?

- Vs rasterio and geopandas: https://geoutils.readthedocs.io/en/latest/about_geoutils.html#comparison-rasterio-geopandas
- Rioxarray is just an xarray accessor → not many added functionalities

Why the need for xDEM?

- Many libraries re-inventing the wheel



DOI: 10.5281/zenodo.5733347



David Shean
dshean

demcoreg

Python and shell scripts for co-registration of rasters, specifically horizontal and vertical alignment of digital elevation models (DEMs).



Bob McNabb
iamdonovan

pyddem

This repository hosts the workflows for creating stacks of DEMs, processing time series of DEMs, and extracting relevant glaciological parameters such as volume changes.



GeoUtils



Andrew Tedstone
atedstone



Amaury Dehecq
adehecq

This repository has been archived by the owner on Feb 19, 2021. It is now read-only.

Hands-on exercises – Get code



Advanced

- Clone repository: ``git clone https://github.com/GlacioHack/xdem-demo.git -b 2024_ige``
- Download data ``cd xdem-demo; bash download_data.sh``

Beginner

- Download code folder: https://github.com/GlacioHack/xdem-demo/archive/refs/heads/2024_ige.zip
- Download data: <https://filesender.renater.fr/?s=download&token=bc73932e-4999-4237-8554-2e1a8862fcd2>
- Extract and save under folder “data” in the code folder.

Hands-on exercises - Install

1. Install

```
`mamba env create --file environment.yml`
```

Or

```
`mamba create xdem=0.0.19 jupyter folium mapclassify -n xdem-demo`
```

2. Open jupyter

```
`conda activate xdem-demo`
```

```
`jupyter notebook`
```



Hands-on exercises - GeoUtils

Beginners:

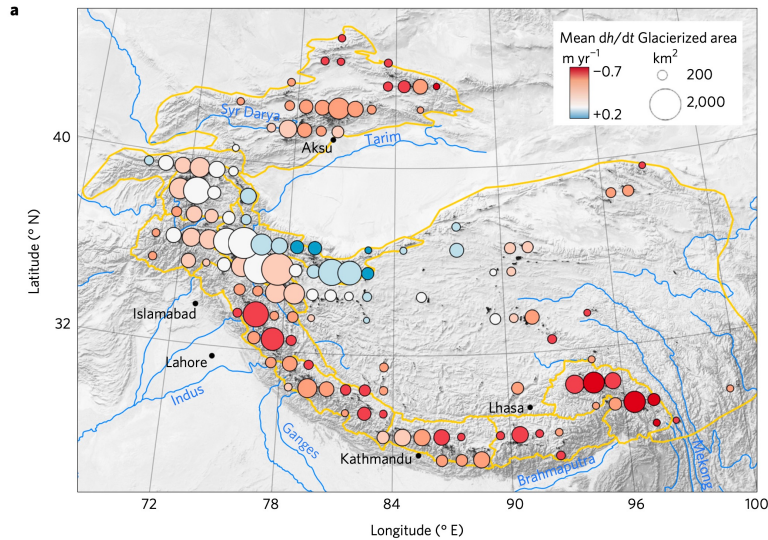
- notebooks/beginners/intro_geoutils.ipynb

Intermediate/advanced:

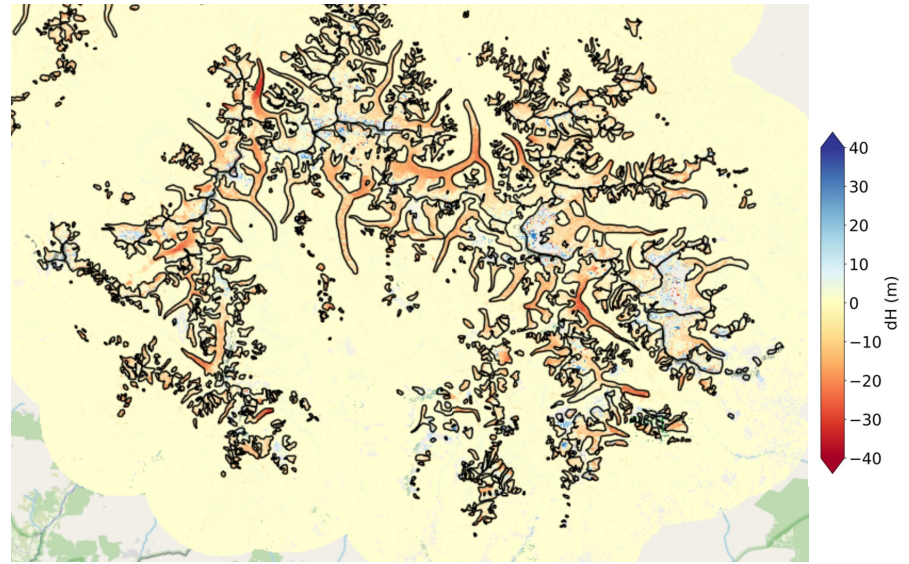
- Go through the latest doc, from Features/Referencing -> Distance operations:
<https://geoutils.readthedocs.io/en/stable/georeferencing.html>
- notebooks/advanced/intro_geoutils.ipynb

DEM differencing

- $dH = DEM(t1) - DEM(t2)$
- Useful for observing changes in surface elevation (glacier thinning, erosion...)



Brun et al. (2017)



Hugonnet et al. (2021)

DEM differencing

Problem 1: Both DEMs need to be:

- with the same vertical datum
- In the same projection
- On the same grid

Solutions:

- Vertical datum correction
- Cropping
- Reprojecting

DEM differencing

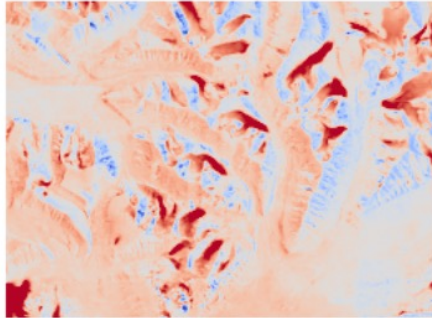
Problem 2: Both DEMs need to be aligned/coregistered:

- Horizontal/vertical shift
- Tilt or more complex non linear shifts

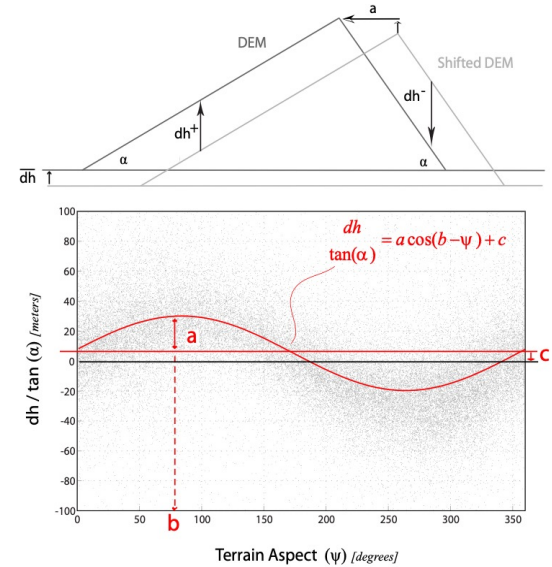
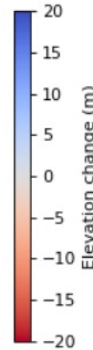
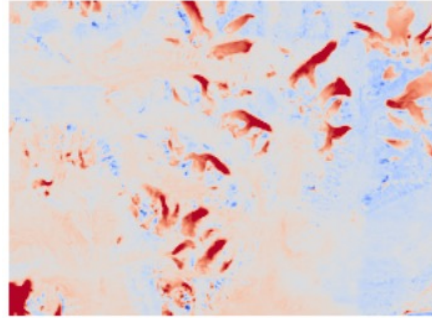
Solution: Coregistration

- Horizontal shift → Nuth & Kääb (2011) method
- Bias correction
- Iterative closest Point Approach

Before coregistration. NMAD=3.4 m



After coregistration. NMAD=2.5 m



Nuth & Kääb algorithm

More info: <https://xdem.readthedocs.io/en/stable/coregistration.html>

Hands-on exercises - xDEM



Beginners:

- [notebooks/beginners/mass_balance_mera.ipynb](#)

Intermediate/Advanced:

- [notebooks/advanced/mass_balance_mera.ipynb](#)

Not interested in DEMs:

- [notebooks/makina/vegetation_paris.ipynb](#)

Experience: Python for geospatial analysis training

<https://makina-corpus.com/formations/formation-sig-cartographie/formation-python-pour-lanalyse-geospatale>

- Very interesting but very “vector oriented”
- Good to know the state-of-the-art in Python geospatial

Highlights:

- `geodataframe.explore` !
- contextily
- Folium
- KeplerGL (only for Vectors but more features than Folium)
- streamlit, shiiny -> For interactive plots with selection/filtering etc commands

Install xdem with Anaconda

- Installing Anaconda Navigator: <https://docs.anaconda.com/anaconda/install/windows/>
- Follow the instructions here: <https://docs.anaconda.com/anaconda/navigator/getting-started/>
- With the specific instructions below (and screenshot on next page):
 - 1) Create a new conda environment (e.g. “xdem”), with the latest Python version
 - 2) In the channels button, add ‘conda-forge’ and remove ‘default’
 - 3) Next to “channel” make sure you select “all”
 - 4) Search for ‘xdem’ in right bar
 - 5) Select ‘xdem’ and click “apply”

Install xdem with Anaconda Navigator

1) Click 'Create' in the bottom left.

2) Click 'Channels' in the top filter bar.

3) Click the search bar.

4) Click the 'xdem' search result.

5) Click the 'Apply' button at the bottom right.

Name	Description	Version
base (root)		
ASPy		
base		
docs		
kh9		
pygeo		
ray		
sglue		
work		
xdem		
xdem-demo		
xdem-dev		

1 package available matching "xdem" 1 package selected

Apply Clear